

Trabalho Final

Implementação do Protocolo Go-Back-N em Java via UDP

Disciplina:	Redes de Computadores
Professor:	Flavio Barbieri Gonzaga
Datas de apresentação:	02 e 03 de julho de 2026
Valor:	10,0 pontos (peso: 2,5 na média final)
Modalidade:	Individual ou em dupla

1. Contexto e Motivação

A transferência confiável de dados é um dos desafios centrais em redes de computadores. O livro *Computer Networking: A Top-Down Approach* (Kurose & Ross) apresenta, no Capítulo 3, uma sequência progressiva de protocolos de transferência confiável — desde o modelo simplificado **rdt 1.0** até os protocolos com janela deslizante **Go-Back-N (GBN)** e **Selective Repeat (SR)**. Este trabalho foca na implementação prática do **Go-Back-N**, levando os conceitos teóricos do livro para um programa Java funcional que transfere arquivos reais entre dois hosts via sockets UDP.

Como o UDP não oferece garantias de entrega, sequenciamento ou controle de fluxo, toda a lógica de confiabilidade deverá ser construída pela própria aplicação, exatamente como descrito no modelo de máquina de estados finita (FSM) do GBN.

2. O Protocolo Go-Back-N (GBN)

2.1 Princípio de funcionamento

No GBN, o emissor pode ter até **N** pacotes não reconhecidos (não confirmados) em trânsito simultaneamente — a chamada *janela de transmissão* de tamanho **N**. Os pacotes são numerados com um número de sequência de k bits, o que limita o espaço de numeração a 2^k valores. O tamanho máximo da janela é, portanto, $N \leq 2^k - 1$.

O receptor do GBN adota uma política simples: aceita apenas pacotes *em ordem*. Se um pacote fora de ordem for recebido, ele é descartado e o receptor reenvia o ACK do último pacote recebido corretamente. O emissor, ao expirar um temporizador (*timeout*), retransmite **todos** os pacotes dentro da janela a partir do pacote não confirmado mais antigo — daí o nome *Go-Back-N*.

2.2 Máquinas de Estado Finitas (FSMs)

A implementação deve seguir fielmente as FSMs apresentadas no livro (Figuras 3.20 e 3.21 da 8ª edição). O emissor possui dois estados principais:

- **Aguardando chamada de cima (Wait for call from above):** janela com espaço disponível; aceita novos segmentos da camada de aplicação.
- **Janela cheia / temporizador ativo:** transmite pacotes dentro da janela e gerencia timeout e ACKs recebidos.

O receptor possui um único estado e responde com ACK cumulativo para cada pacote recebido em ordem.

2.3 Variáveis do protocolo

Variável	Descrição
base	Número de sequência do pacote mais antigo não confirmado
nextseqnum	Próximo número de sequência a ser usado
N (windowSize)	Tamanho da janela de transmissão (configurável pelo usuário)
expectedseqnum	Número de sequência esperado pelo receptor (somente em ordem)

3. Especificação do Trabalho

3.1 Visão geral

O trabalho consiste em implementar dois módulos Java independentes — **Receptor** e **Emissor** — que se comunicam exclusivamente via sockets UDP, simulando a transferência confiável de um arquivo arbitrário utilizando o protocolo Go-Back-N.

3.2 Módulo Receptor

O Receptor deve ser inicializado antes do Emissor e ficar aguardando conexões em uma porta UDP configurável (sugestão: porta **5000**). Ao receber o primeiro pacote de controle (handshake), o Receptor extrai os parâmetros da sessão e começa a receber os dados.

O Receptor deve:

- Aguardar na porta UDP configurada por um datagrama inicial de controle contendo os parâmetros da sessão (probabilidade de perda, nome/path do arquivo de destino, tamanho do arquivo).
- Implementar a FSM do receptor GBN: aceitar apenas pacotes com `seqnum == expectedseqnum`; descartar pacotes fora de ordem e reenviar o último ACK enviado.
- Simular perda de pacotes: com base na probabilidade recebida, descartar pacotes de forma aleatória, sem enviar ACK, forçando retransmissão pelo emissor.
- Salvar o arquivo recebido no path absoluto especificado pelo emissor.
- Ao final da transferência, exibir estatísticas: total de pacotes recebidos, total de pacotes descartados (simulados como perdidos) e taxa de perda efetiva.

3.3 Módulo Emissor

O Emissor é iniciado via linha de comando com os seguintes argumentos obrigatórios:

```
java Emissor <arquivo_origem> <IP_destino>:<path_destino> <tamanho_janela> <prob_perda>
```

Exemplo de uso:

```
java Emissor /home/alice/foto.jpg 192.168.0.10:/tmp/foto_recebida.jpg 8 0.10
```

O parâmetro `<prob_perda>` é um valor real entre 0,0 e 1,0 (p. ex.: 0,10 = 10% de probabilidade de perda), que será enviado ao Receptor no pacote de handshake inicial.

O Emissor deve:

- Dividir o arquivo de origem em segmentos de tamanho fixo (sugestão: 1024 bytes de payload), numerados sequencialmente.
- Implementar a FSM do emissor GBN com janela deslizante de tamanho N.
- Iniciar um temporizador único para o pacote mais antigo não confirmado (*base*).

- Ao receber um ACK cumulativo de número n , avançar a base da janela para $n+1$ e reiniciar/cancelar o temporizador conforme a FSM.
- Em caso de timeout, retransmitir todos os pacotes de *base* até *nextseqnum* – 1.
- Enviar um pacote de controle de encerramento (FIN) ao final da transmissão.
- Exibir progresso em tempo real: número de pacotes enviados, ACKs recebidos, retransmissões e taxa de throughput estimada.

3.4 Formato do Datagrama

Cada datagrama UDP deve encapsular um segmento com cabeçalho definido pela dupla/aluno. Sugere-se, no mínimo, os seguintes campos:

Campo	Tamanho sugerido	Descrição
tipo	1 byte	0=DATA, 1=ACK, 2=HANDSHAKE, 3=FIN
num_seq	4 bytes (int)	Número de sequência do pacote
num_ack	4 bytes (int)	Número de confirmação (somente em ACKs)
tamanho_dados	2 bytes (short)	Quantidade de bytes válidos no payload
dados	até 1024 bytes	Payload (bytes do arquivo)

4. Simulação de Perda de Pacotes

Como o ambiente de testes é uma rede local (LAN), a probabilidade real de perda de pacotes é praticamente nula. Para validar o comportamento do GBN, o **Receptor** deve simular perdas de forma aleatória, descartando pacotes de dados sem enviar ACK, como se o pacote nunca tivesse chegado.

A lógica de descarte deve ser baseada em geração de número aleatório: para cada pacote recebido (e em ordem), o Receptor sorteia um valor $r \in [0,1)$. Se $r < p$ (onde p é a probabilidade de perda configurada), o pacote é descartado silenciosamente. Ao final da transferência, a taxa de perda efetiva deve tender à probabilidade configurada à medida que o número de pacotes transferidos aumenta (Lei dos Grandes Números).

Atenção: a simulação deve atuar *somente* sobre pacotes de dados recebidos corretamente em ordem. Pacotes já fora de ordem são descartados pela própria lógica do GBN e não devem ser contabilizados como perdas simuladas.

5. Requisitos Técnicos

#	Requisito	Tipo
R1	Implementação exclusivamente em Java (sem frameworks externos além do JDK padrão)	Obrigatório
R2	Uso exclusivo de sockets UDP (DatagramSocket / DatagramPacket)	Obrigatório
R3	Lógica GBN fiel às FSMs do Kurose & Ross (janela, timeout, retransmissão)	Obrigatório
R4	Transferência correta de arquivos binários (imagens, PDFs, executáveis)	Obrigatório
R5	Parâmetros de execução via linha de comando conforme especificado	Obrigatório
R6	Exibição de estatísticas ao final (pacotes enviados, retransmitidos, perdas)	Obrigatório

R7	Código organizado, comentado e com README explicando como compilar e executar	Obrigatório
R8	Variação do tamanho da janela N e análise do impacto no tempo de transferência	Desejável
R9	Verificação de integridade do arquivo recebido (ex.: comparação de hash MD5/SHA-1)	Desejável

6. Entrega e Apresentação

A entrega deverá ser realizada via repositório Git (GitHub, GitLab ou similar), com link enviado pelo sistema da instituição até às **23h59 do dia 01 de julho de 2026**. O repositório deve conter:

- Código-fonte completo dos módulos Emissor e Receptor.
- Arquivo README.md com instruções de compilação, execução e exemplos de uso.
- Relatório técnico em PDF (mínimo 3 páginas) descrevendo as decisões de projeto, dificuldades encontradas, testes realizados e análise dos resultados (incluindo gráficos ou tabelas comparando diferentes valores de N e probabilidades de perda).

As apresentações ocorrerão em laboratório nos dias **02 e 03 de julho de 2026**, com duração de aproximadamente 15 minutos por dupla/aluno. Durante a apresentação, o programa deverá ser executado ao vivo demonstrando a transferência de um arquivo com pelo menos 1 MB, com probabilidade de perda de 10%, e o avaliador poderá interromper e questionar sobre qualquer parte do código.

7. Critérios de Avaliação

Critério	Peso
Correção do protocolo GBN (janela, timeout, retransmissão)	30%
Simulação de perda e estatísticas ao final	15%
Transferência correta de arquivos binários de diferentes tipos	15%
Qualidade do código (organização, comentários, README)	15%
Relatório técnico (clareza, análise e resultados)	10%
Apresentação oral e domínio do código	10%
Requisitos desejáveis (hash de integridade, análise de N)	5%

8. Dicas de Implementação e Referências

Abaixo seguem orientações práticas que podem facilitar o desenvolvimento:

- **Temporizador:** utilize uma thread dedicada ou ScheduledExecutorService para disparar o timeout. Cancele o timer ao receber ACK que avança a base; reinicie-o se ainda houver pacotes não confirmados.
- **Sincronização:** o emissor terá ao menos duas threads concorrentes (envio e recepção de ACKs). Use synchronized ou estruturas java.util.concurrent para proteger as variáveis compartilhadas (base, nextseqnum, buffer de pacotes).
- **Buffer circular:** armazene os últimos N pacotes enviados para possibilitar retransmissão sem reler o arquivo do disco.
- **Serialização:** utilize ByteBuffer para montar e desmontar o cabeçalho dos datagramas de forma portátil.

- **Teste incremental:** comece com `prob_perda = 0.0` para validar a transferência básica antes de ativar a simulação de erros.
- **Verificação:** compare o hash MD5 do arquivo original com o recebido usando `MessageDigest` da API Java padrão.

Referências Bibliográficas

KUROSE, James F.; ROSS, Keith W. **Redes de Computadores: Uma Abordagem Top-Down**. 8. ed. São Paulo: Pearson, 2021. Capítulo 3, Seções 3.4 e 3.4.3.

TANENBAUM, Andrew S.; WETHERALL, David. **Redes de Computadores**. 5. ed. São Paulo: Pearson, 2011. Capítulo 3.